

Putting It All Together

Learning Notes — The Complete Web Request Journey

"When you type a URL and press Enter, over a dozen interconnected systems activate in milliseconds to deliver that web page. This room shows you every single one of them."



1. The Big Picture — How Everything Connects

This capstone room synthesises every concept covered in the TryHackMe Pre-Security learning path: DNS, HTTP, HTML, JavaScript, networking, and web server infrastructure. When you type a URL into your browser and press Enter, a complex chain of events unfolds involving a dozen or more separate systems — all within milliseconds.

The Core Summary

To access a website: your computer uses DNS to find the server's IP address, then communicates using HTTP to request the content, and the web server returns HTML, CSS, JavaScript, and images which your browser renders into the page you see. Between the user and the content lie additional infrastructure layers — CDN, WAF, load balancer, application server, and database — each performing a specific role.

1.1 The Three Phases of Every Web Request

Phase	What Happens	Systems Involved
1. Discovery	Find the server's IP address	Local DNS cache → Recursive DNS server → Root DNS → TLD DNS → Authoritative DNS
2. Edge	Protect and distribute the request	CDN (static content) → WAF (security filter) → Load Balancer (distribute to healthy server)

3. Serve	Generate and return the content	Web server software → Virtual hosts → Application code → Database → HTTP response → Browser renders HTML/CSS/JS
-----------------	---------------------------------	---

2. The Step-by-Step Journey — Requesting tryhackme.com

This is the complete, ordered sequence of every step that occurs when you type tryhackme.com into your browser and press Enter. This is the exact flow tested in the room's drag-and-drop ordering challenge.

1 Check Local DNS Cache

Before making any network requests, your computer checks its local DNS cache. If it has recently looked up tryhackme.com and the TTL hasn't expired, it uses the cached IP address immediately — no network query needed. Speed: microseconds.

From: Browser (you press Enter) → **To:** Local DNS cache (hit → done; miss → Step 2)

2 Query Recursive DNS Server

If the local cache misses, your computer sends a DNS query to the Recursive DNS Server — typically provided by your ISP, or a custom one like 8.8.8.8 (Google) or 1.1.1.1 (Cloudflare). The recursive server also checks its own cache first.

From: Your computer → **To:** Recursive DNS Server (your ISP or 8.8.8.8 / 1.1.1.1)

3 Recursive Server Queries Root DNS

If the recursive server's cache misses too, it queries one of the 13 Root DNS Server sets. The root server doesn't know the IP of tryhackme.com, but it knows which server handles .com TLD domains. It replies: 'Ask the .com TLD server.'

From: Recursive DNS Server → **To:** Root DNS Servers (a–m.root-servers.net)

4 Recursive Server Queries TLD DNS

The recursive server queries the .com TLD nameserver. The TLD server knows which authoritative nameserver is responsible for tryhackme.com. It replies: 'Ask ns1.tryhackme.com — that's the authoritative server for this domain.'

From: Root DNS Server reply → **To:** TLD DNS Server (.com nameservers)

5 Authoritative DNS Returns IP Address

The recursive server queries tryhackme.com's own authoritative nameserver (ns1.tryhackme.com). This server holds the actual DNS zone records. It looks up the A record and returns the definitive answer: 'tryhackme.com → 104.26.10.229'. The recursive server caches this for the TTL duration and returns it to your computer.

From: TLD DNS Server reply → **To:** Your computer now has the IP address: 104.26.10.229

6 Request Passes Through WAF

Your HTTP/HTTPS request travels to the target IP. Before reaching the web server, it passes through a Web Application Firewall (WAF). The WAF inspects the request for malicious patterns — SQL injection, XSS, bot signatures, excessive request rates (rate limiting). Malicious requests are dropped here. Legitimate requests proceed.

From: Your browser (HTTP GET request) → **To:** WAF — first line of application-layer defence

7 Request Reaches Load Balancer

The WAF-cleared request hits the Load Balancer. The load balancer distributes incoming traffic across multiple backend web servers using algorithms (round-robin, least connections, IP hash). It also runs periodic health checks against each server — unhealthy servers are removed from rotation until they recover.

From: WAF (request cleared) → **To:** Load Balancer → one of several backend web servers

8 Web Server Receives the Request

The load balancer forwards the request to a specific web server. The web server software (Nginx, Apache, IIS, Node.js) receives the HTTP GET request. It uses Virtual Hosts to identify which website is being requested (via the Host header) — one server can host dozens of websites simultaneously.

From: Load Balancer → **To:** Web server software (Nginx / Apache / IIS)

9 Application Queries the Database

For dynamic content (user profiles, blog posts, product listings), the web server passes the request to the application code (Python, PHP, Node.js). The application constructs a database query (MySQL, MongoDB, PostgreSQL) to fetch the required data. The database returns the data, the application builds the HTML response.

From: Web server → application code → **To:** Database → application generates dynamic HTML response

10 Response Returned — Browser Renders

The web server sends the HTTP response back through the load balancer and WAF to your browser. The response contains status code (200 OK), headers, and the HTML body. Your browser parses the HTML, requests any additional resources (CSS, JS, images — potentially served from CDN), and renders the complete page.

From: Web server HTTP response → **To:** Your browser — page fully rendered and displayed

The Drag-and-Drop Challenge Answer

The correct ordering for the room's 'Request tryhackme.com in your browser' challenge: (1) Local DNS Cache check → (2) Recursive DNS Server → (3) Root DNS → (4) TLD DNS → (5) Authoritative DNS returns IP → (6) WAF filters request → (7) Load Balancer distributes → (8) Web Server receives → (9) Application + Database → (10) Response rendered by browser.

3. The Infrastructure Components

Between the user and the web server content lie several infrastructure components. Each has a specific purpose — performance, reliability, or security. Understanding all of them is essential for both building scalable web applications and assessing their security.



CDN — Content Delivery Network — Speed up static content delivery by serving from the nearest geographic server

- A CDN is a globally distributed network of servers (edge nodes) that cache copies of static assets — JavaScript, CSS, images, videos, fonts.
- When a user requests a static file, the CDN serves it from the edge node geographically closest to the user — reducing latency and the distance data travels.
- Example: a user in Mumbai accessing tryhackme.com gets images from a CDN node in Mumbai, not from a server in the UK.
- CDNs dramatically reduce load on the origin server — static files don't hit the origin at all.
- CDNs also provide resilience — if the origin server goes down, cached static content remains accessible.
- Popular CDNs: Cloudflare, Amazon CloudFront, Fastly, Akamai, Azure CDN.
- Answer: 'What hosts static files and speeds up a client visit?' → CDN

Security Note: CDN edge nodes cache content, but a misconfigured CDN can serve stale or poisoned content. CDN origin pull can expose origin server IPs if not properly masked. Attackers target CDN configuration to serve malicious scripts under trusted domains.



Load Balancer — Distribute traffic across multiple servers for availability and scalability

- A load balancer sits in front of multiple identical web servers and distributes incoming requests across them.
- Purpose 1 — High availability: if one server fails, others continue serving requests. No single point of failure.
- Purpose 2 — Scalability: as traffic grows, add more servers behind the load balancer without any change to the domain or IP.
- Load balancing algorithms: Round Robin (sequential), Least Connections (send to least busy), IP Hash (sticky sessions based on client IP).
- Health checks: load balancers periodically send test requests (HTTP/TCP probes) to each backend server. If a server fails health checks, it is removed from rotation until it recovers.
- Answer: 'What does a load balancer perform to ensure a server is alive?' → Health Check
- Common load balancers: Nginx, HAProxy, AWS ELB/ALB, Azure Load Balancer, F5.

Security Note: Load balancers can themselves be DDoS targets. Session persistence (sticky sessions) can cause overloading of individual servers. Load balancer logs reveal the actual backend server IPs, which attackers seek to bypass the load balancer and attack directly.



WAF — Web Application Firewall — Inspect and filter HTTP/HTTPS requests to block web-layer attacks

- A WAF sits between the user's request and the web server, inspecting all HTTP/HTTPS traffic at OSI Layer 7 (Application layer).
- The WAF analyses requests for known attack patterns: SQL injection strings, XSS payloads, path traversal sequences, malicious file uploads.
- Rate limiting: WAFs block IPs that send an excessive number of requests per second — mitigating DDoS and brute force attacks.
- Bot detection: WAFs verify whether requests come from real browsers or automated bots (by analysing headers, TLS fingerprints, and behaviour).
- Malicious requests are dropped before they reach the application — the web server never sees them.
- Answer: 'What can be used to help against the hacking of a website?' → WAF
- Popular WAFs: Cloudflare WAF, AWS WAF, ModSecurity, Imperva, Akamai Kona.

Security Note: WAFs can be bypassed by attackers who encode payloads, use unusual HTTP methods, or exploit WAF rule gaps. WAF bypass techniques are a major area of web application penetration testing. A WAF is a defence-in-depth layer — not a replacement for secure code.

4. How Web Servers Work

A web server is software that listens for incoming HTTP connections and responds with web content. It is not a specific machine — any computer running web server software is a web server. Understanding how web server software works is fundamental to web security assessment.

4.1 Common Web Server Software

Software	Key Characteristic	Common Usage
Nginx	High performance, event-driven, low memory	High-traffic websites, reverse proxy, load balancing, static file serving. Most popular web server globally.
Apache	Module-based, highly configurable (.htaccess)	Shared hosting, PHP applications, flexible configuration. Widely used but higher memory footprint than Nginx.
IIS	Windows-native, integrates with .NET/ASP.NET	Microsoft technology stacks — .NET web applications, Windows Server environments, SharePoint.
Node.js	JavaScript runtime — code runs directly in server	Real-time applications (chat, live updates), APIs, single-page app backends. Not traditional web server software but widely used.

Web Server Security Implication

The Server response header (e.g. Server: nginx/1.19.0) reveals the web server software and version. Attackers use this for targeted CVE research — every version has a known vulnerability list. Best practice: suppress or obscure the Server header in production configuration.

4.2 Virtual Hosts — Hosting Multiple Sites on One Server

Web servers can host multiple different websites on a single machine using a feature called Virtual Hosts. The web server reads the Host header from each incoming HTTP request and serves the matching website from its configuration.

Host Header Received	Website Served by Virtual Host
Host: one.com	Serves /var/www/one.com — the content for website one.com
Host: two.com	Serves /var/www/two.com — completely different website on same server
Host: three.com	Serves /var/www/three.com — third website, same server, same IP
Host: (no match)	Serves the default virtual host — often a placeholder or error page

Virtual Hosts Answer

The room asks: 'What does web server software use to host multiple sites?' → Virtual Hosts. This is how shared hosting works — hundreds of websites on one server, each served correctly based on the Host header. Host header injection attacks exploit web servers that trust the Host header for routing or link generation.

4.3 Static vs Dynamic Content

Feature	Static Content	Dynamic Content
What it is	Pre-built files served as-is — same for every user	Generated on-demand per request — content varies per user/context
Examples	HTML files, CSS, JavaScript, images, videos, PDFs	User dashboards, news feeds, search results, personalised pages
Processing	Web server reads the file from disk and serves it directly	Web server passes request to app code → app queries DB → builds HTML
Speed	Very fast — no processing required	Slower — database queries and processing add latency

CDN-cacheable	Yes — ideal CDN candidates	Usually no — content is user-specific or real-time
Answer	(Not a specific room question)	'What is the name for content that can change?' → Dynamic

Does the Client See Backend Code?

No — Nay. Backend code (Python, PHP, Node.js, Ruby) runs on the server and is never sent to the browser. Only the output it generates (HTML, JSON) is sent. This is fundamental — if a PHP file is served raw instead of executed (server misconfiguration), the source code including database credentials is exposed to every user.

5. Databases

Databases are the persistent storage layer of web applications. Almost every dynamic website uses a database to store and retrieve data — user accounts, posts, products, orders, settings. The web server communicates with the database via the application code, never directly from the browser.

5.1 Types of Databases

Database	Type	Characteristics & Common Use
MySQL	Relational (SQL)	Most popular open-source relational DB. Uses structured tables with rows/columns. Standard for PHP/WordPress/LAMP stack applications.
MSSQL	Relational (SQL)	Microsoft SQL Server. Enterprise-grade, integrates tightly with .NET and Windows environments.
PostgreSQL	Relational (SQL)	Advanced open-source relational DB. Strong standards compliance, excellent for complex queries and high-integrity data.
MongoDB	NoSQL (Document)	Stores data as JSON-like documents. Flexible schema — ideal for rapidly changing data structures and large-scale applications.
SQLite	Relational (File)	Single-file database. No server required. Used in mobile apps, embedded systems, and simple applications.
GraphQL	Query Language	Not a database itself — a query language and API standard for requesting precisely the data you need from a backend.

SQL Injection — The Database Attack

SQL injection is the most prevalent and impactful database attack. It occurs when user-supplied input is concatenated directly into SQL queries without sanitisation — allowing

attackers to modify the query structure. A successful SQL injection can extract entire databases, bypass authentication (login as admin without a password), modify or delete data, and in some cases execute OS commands. Parameterised queries / prepared statements completely prevent SQL injection.

6. All Room Answers — Quick Reference

Question	Answer
What can be used to host static files and speed up a client's visit?	CDN
What does a load balancer perform to make sure a host is still alive?	health check
What can be used to help against the hacking of a website?	WAF
What does web server software use to host multiple sites?	Virtual Hosts
What is the name for the type of content that can change?	Dynamic
Does the client see the backend code? (Yay/Nay)	Nay

7. The Complete Pre-Security Path — What You've Learned

This room is the capstone of the TryHackMe Pre-Security learning path. Every concept covered in the preceding rooms contributes to understanding the complete web request journey. Here is how every room connects:

Room	Topic	Role in the Full Journey
What is Networking?	IP & MAC addresses, Ping	Every web request travels over a network. IP addresses identify the destination. Ping verifies connectivity.
Intro to LAN	Switches, Routers, Subnets	Your request travels through your local LAN (switch) to your router before reaching the internet.
OSI Model	7-layer networking model	Every request involves all 7 layers — from physical signals to the application-level HTTP message.

Packets & Frames	TCP, UDP, Ports	Web requests use TCP (reliable delivery) on port 80 (HTTP) or 443 (HTTPS) to transfer data.
Extending Your Network	Firewalls, VPN, NAT	Your router performs NAT. Firewalls and WAF filter traffic. VPNs encrypt the entire connection.
DNS in Detail	DNS records & resolution	Steps 1–5 of the journey: translating tryhackme.com to an IP address via the DNS hierarchy.
HTTP in Detail	Requests, responses, headers	The HTTP GET request and response are the actual messages exchanged between browser and server.
How Websites Work	HTML, JavaScript, CSS	The HTML/CSS/JS returned by the server is what the browser renders into the visible web page.
Putting It All Together	Complete synthesis	Connects all of the above into the full end-to-end journey — DNS → WAF → LB → Server → DB → Browser.

8. Security Implications — The Attack Surface of the Full Journey

Every Step Is an Attack Opportunity

The 10-step journey from browser to web page is not just an engineering story — it is a map of the attack surface. Each component that can be compromised, misconfigured, or manipulated represents an entry point for attackers. Understanding the full journey from both directions — user and attacker — is the foundation of web security work.

Step	Attack Vector	Impact & Defence
DNS	DNS Cache Poisoning / Spoofing	Redirects users to malicious servers. Defence: DNSSEC, DNS-over-HTTPS.
DNS	Subdomain Takeover	Dangling CNAME to unclaimed service — attacker serves content under trusted domain.
CDN	Cache Poisoning	Injecting malicious content into CDN cache — served to all users. Defence: cache validation headers.
WAF	WAF Bypass	Encoding payloads to evade WAF rules. WAF is a layer, not a cure — secure code is mandatory.

Load Balancer	DDoS / Health Check Flooding	Overwhelming backend servers or targeting health check endpoints. Defence: rate limiting, DDoS mitigation.
Web Server	Server Version Exposure	Server header reveals version → targeted CVE exploitation. Defence: suppress server headers.
Virtual Host	Host Header Injection	Manipulating Host header to access other virtual hosts or poison cache. Defence: strict Host validation.
Application	SQL Injection / XSS / CSRF	All originate from unsanitised user input in application code. Defence: parameterised queries, CSP, CSRF tokens.
Database	Credential Exposure	DB credentials in source code, config files, or environment variables. Defence: secrets management.
Response	Sensitive Data in Source	Credentials in HTML comments, JS files. Defence: code review, pre-deployment security scanning.

9. Master Glossary — All Key Terms

Term	Definition
DNS	Domain Name System — translates domain names (tryhackme.com) to IP addresses (104.26.10.229).
DNS Cache	Local store of recent DNS lookups — checked first before any network query to save time.
Recursive DNS	DNS server that does the full lookup chain on your behalf (your ISP or 8.8.8.8/1.1.1.1).
Root DNS	Top-level DNS servers that direct queries to the correct TLD server (13 sets: a–m.root-servers.net).
TLD DNS	Top-Level Domain nameservers — direct to the authoritative server for the specific domain (.com, .org, .uk).
Authoritative DNS	The definitive DNS server for a domain — holds the actual A, MX, TXT records.
HTTP	HyperText Transfer Protocol — the communication protocol browsers and servers use to exchange web content.
HTTPS	HTTP with TLS encryption — encrypts the request/response so interceptors cannot read the content.
CDN	Content Delivery Network — globally distributed servers caching static files close to users for speed.

Load Balancer	Distributes incoming requests across multiple backend servers. Performs health checks to remove failed servers.
Health Check	Periodic test sent by load balancer to verify a server is alive and responding correctly.
WAF	Web Application Firewall — inspects HTTP traffic for attack patterns and drops malicious requests.
Rate Limiting	WAF/load balancer feature — caps the number of requests from one IP per second to prevent DDoS/brute force.
Web Server	Software (Nginx, Apache, IIS) that listens for HTTP connections and serves web content.
Virtual Host	Web server configuration that maps a Host header value to a specific website directory on the server.
Static Content	Pre-built files served as-is — HTML, CSS, JS, images. Same for every user. Cacheable by CDN.
Dynamic Content	Content generated per-request by application code — personalised, real-time. Requires database queries.
Database	Persistent data storage for web applications. Examples: MySQL, MongoDB, PostgreSQL, MSSQL.
SQL	Structured Query Language — used to interact with relational databases (SELECT, INSERT, UPDATE, DELETE).
SQL Injection	Attack where unsanitised user input modifies SQL query structure — bypasses auth, extracts data.
Front End	Client-side code (HTML/CSS/JS) rendered by the browser. Always publicly readable.
Back End	Server-side code (Python/PHP/Node) running on the server. Never sent to the browser — Nay.
OSI Model	7-layer networking framework. Web requests traverse all 7 layers — Physical through Application.
TCP	Transport layer protocol used by HTTP/HTTPS — reliable, connection-oriented (port 80/443).
TLS/SSL	Transport Layer Security — encryption protocol used by HTTPS to protect data in transit.

9.1 Where to Go Next

- **Offensive:** Start the Jr Penetration Tester learning path — apply every concept covered here offensively.
- **Defensive:** Start the SOC Level 1 path — monitor, detect, and respond to attacks against these exact systems.

- **Web Security:** Complete the OWASP Top 10 room — the most critical web vulnerabilities in applications like these.
- **Essential Tool:** Take the Burp Suite Basics room — the primary tool for intercepting and testing web requests.
- **Network Security:** Study the Network Security module — firewalls, IDS/IPS, traffic analysis for the network layer.
- **Certification:** Work towards eJPT (Entry-level Junior Penetration Tester) certification as a formal milestone.

You Have Completed the Pre-Security Path

Completing this room marks the end of the TryHackMe Pre-Security learning path. You now understand: how devices communicate over networks (OSI, TCP/IP, DNS), how web content is delivered and protected (HTTP, CDN, WAF, load balancers, web servers, databases), and how websites are built and where they are vulnerable (HTML, JavaScript, sensitive data exposure, HTML injection). This is the foundation everything else in cyber security is built on.

Source: TryHackMe — Putting It All Together Room | tryhackme.com/room/puttingitaltogether